FINE-STACK: Flexible architecture for low-power lossy IoT Networks with programmablE network STACK and semantic runtime updates

Ahmad Mahmod[a,*], Julien Montavont[a], Thomas Noel[a]

[a]*ICube lab (UMR CNRS 7357), University of Strasbourg, Strasbourg, 67000, France*

Abstract

Low-Power and Lossy Wireless Networks (LLWNs) operate under stringent energy, memory, and bandwidth constraints and rely on inherently unreliable multi-hop links. Despite substantial progress in IoT networking, updating deployed LLWNs often still requires bulky, full-image firmware upgrades, even for minor network stack changes. This practice incurs excessive overhead, complicates reliable delivery over lossy links, and can temporarily disrupt connectivity due to mandatory reboots. We introduce FINE-STACK, a flexible architecture that decomposes the network stack into lightweight virtual functions coordinated by a central orchestrator. By enabling fine-grained composition and semantics-aware management of protocol components, FINE-STACK supports fast and selective over-the-air updates at runtime. It further improves the programmability of the network stack by enabling hybrid deployments that combine local in-network control with optional global optimization. We implemented and evaluated FINE-STACK on a real-world testbed, comparing programmability and performance across conventional deployment models and the hybrid model enabled by our design. The results demonstrate that hybrid deployments provide improved programmability and superior operational performance, while fine-grained modularity of the network stack substantially reduces update size and update time without violating strict memory, power and overhead constraints. Overall, FINE-STACK advances LLWNs toward practical network programmability, narrowing the gap between flexibility and efficiency, and making semantic runtime reconfiguration feasible on highly constrained IoT devices.

Keywords: Hybrid Control Plane, Internet of Things, Low-power Lossy Wireless

*Corresponding author

Email address: mahmod@unistra.fr (Ahmad Mahmod)

Networks, Wireless Sensor Networks, Programmable Networks, Lightweight Virtualization.

1. Introduction

The Internet of Things (IoT) is a computing paradigm that integrates physical objects—such as sensors, actuators, and embedded devices—with networking and processing capabilities to monitor, interact with, and influence the surrounding environment [1]. Low-Power and Lossy Wireless Networks (LLWNs) represent a fundamental component of the IoT ecosystem, comprising devices constrained in energy, processing power, and memory to maintain low cost. These networks also face communication constraints, including limited range and bandwidth, which make them inherently multi-hop and lossy. LLWNs support a wide range of applications, from environmental monitoring and early disaster detection to industrial automation and smart city infrastructure [2].

Given the long operational lifetime expected from LLWNs and their exposure to dynamic environmental conditions, software updates are essential for maintaining functionality and extending device longevity. In addition to prolonging lifetime, updates reduce electronic waste (E-waste) and minimize the need for physical interventions. Over-the-Air (OTA) updates provide a practical and cost-effective means of remotely updating devices without physical access, thereby reducing operational and maintenance costs [3].

Most LLWN operating systems hardcode the network stack within the system, requiring a full (or sometimes partial) firmware image replacement even for minor modifications such as parameter tuning [4]. Although this approach is simple, full-image replacement is inherently inefficient and semantically blind since updates are handled purely as binary images without understanding their intent or affected components. The large binary size must be transmitted over bandwidth-limited multi-hop networks, which significantly increases transmission time and failure probability while also congesting the network. In addition, installation requires a reboot, resulting in loss of network state and triggering a new convergence process that leads to connectivity interruptions [5]. Since most software updates in LLWNs target the network stack for performance enhancement or reconfiguration [6], reducing the size of the update and enabling semantic-aware updates are essential to achieve fast, transparent, and reliable reconfiguration.

Network functionality in LLWNs is divided into three planes: the *control plane* computes forwarding, scheduling, and policy decisions, the *data plane* enforces these decisions by processing and forwarding packets, and the *management plane* monitors

the network and programs the behavior of the network stack, adapting control and data plane logic to meet global objectives and performance requirements. LLWNs typically employ two control-plane deployment models depending on where the control plane is located. In *centralized deployments*, the control plane is hosted in a central entity that manages distributed data planes across nodes through a backhaul link, leveraging a global network view [7]. By contrast, in *distributed deployments*, the control plane is embedded locally within each node, co-located with the data plane. Control planes collaborate by exchanging control messages, making independent yet collectively sub-optimal decisions due to limited local views [8].

Historically, early LLWN deployments favored the distributed model because establishing a reliable backhaul was challenging. Energy constraints required duty-cycling mechanisms to reduce power consumption, while multi-hop lossy links limited link reliability and availability. Consequently, distributed protocols such as RPL [9] for routing and IEEE 802.15.4 based on CSMA [10] for MAC scheduling were developed. Although these protocols provide autonomy, they can suffer from slow and non-optimal convergence [11]. Updates to the control-plane logic in this model require modifying node-resident control functions, which is costly and therefore done infrequently. More recently, centralized control-plane deployments have been introduced to LLWNs through lightweight adaptations designed for constrained environments [12]. Software-Defined Networking (SDN) is the most representative example, offering globally optimized decision-making. However, centralized models are heavily dependent on reliable backhaul links; under lossy conditions, congestion and disconnections can degrade performance [13]. Network updates in this model mainly take the form of match-action rules sent frequently from the central control plane to distributed data planes.

In summary, no single deployment model performs optimally under all conditions, highlighting the need for a flexible architecture with a modular network stack that can be managed semantically over the air, enabling flexible updates of network planes. Such an architecture should support multiple control-plane configurations and allow seamless runtime switching or reconfiguration between deployments, and even enabling hybrid modes that combine the strengths of distributed and centralized designs while mitigating their respective limitations.

This paper proposes the **Flexible architecture for low-power lossy IoT Networks with programmablE network STACK and semantic runtime updates (FINE-STACK)**, a unified and adaptive network architecture for LLWNs. FINE-STACK enables a wide spectrum of network programmability at runtime, ranging from fine-grained parameter tuning (e.g., adjusting MAC-layer backoff times), to replacing entire protocol mechanisms (e.g., switching from hop-by-hop to

source routing), and up to reconfiguring the control-plane deployment model itself (e.g., transitioning from distributed to hybrid control). The core component of FINE-STACK is a modular network stack in which network functions are implemented as **lightweight Virtual Functions (VFs)**. These VFs provide a fine-grained structure that can be dynamically composed to run different network protocols by chaining sets of VFs. The modular stack is managed through a central orchestrator that enables **selective and semantic over-the-air updates**.

In previous work [14], we presented a proof-of-concept that implemented the UDP protocol using VFs, demonstrating that modular runtime updates can significantly reduce the size and time of network stack updates. Building on this foundation, the present work improves the architecture by introducing semantics-aware updates and evaluates several RIOT/RPL-based control-plane deployment variants, including a hybrid realization that restores routing state after updates.

The main contributions of this paper are as follows:

1. We extend network stack updates in LLWNs with semantic awareness, enabling safe, resilient runtime updates while preserving network state and sustaining performance.
2. We present a practical implementation of both distributed and centralized control-plane deployments with enhanced programmability, and introduce and a hybrid realization where distributed RPL control is complemented by centrally injected state after updates.
3. We conduct a realistic evaluation on a real testbed, comparing the performance of different deployments and demonstrating runtime updates with substantial reductions in update size and time, while respecting the resource constraints of LLWNs.
4. We provide an open source implementation of the proposed FINE-STACK architecture and the RPL non-storing mode on the RIOT operating system.

While this paper focuses on existing routing-oriented control-plane deployments and a state-preserving RPL-based hybrid deployment, the proposed architecture is not limited to these cases and can be extended to other protocols, layers, and future hybrid models. To facilitate further research and reproducibility, we provide open-source access to the FINE-STACK implementation and experimental scripts.

The rest of this paper is organized as follows: Section 2 motivates the proposed architecture while Section 3 introduces the proposed network stack and orchestration framework and explains how different deployments are realized. Section 4 presents a comparative evaluation of the different deployments and enhancements achieved by FINE-STACK. Section 5 reviews related work. Finally, Section 6 concludes the work and outlines future research directions.

2. Motivation

LLWNs evolve after deployment as the density of nodes changes—whether due to expansion of the sensing area or removal of faulty nodes, application requirements shift to enhance performance or support new features, and environmental conditions fluctuate due to moving obstacles, weather variations in outdoor deployments, or changing interference patterns in industrial environments. In such dynamic settings, static and monolithic stacks struggle to maintain efficiency, reliability, and connectivity.

Distributed control-plane deployments are typically realized with monolithic network stacks that tightly couple control and data planes, making modifications coarse-grained and inherently disruptive. In the remainder of this paper, we use RPL [9] as a representative baseline in the motivation and evaluation, since it is the de facto standard routing protocol for LLWNs and exposes typical challenges related to distributed control-plane updates, state maintenance, and reconvergence. In RPL, each node hosts a local control plane and exchanges control messages to construct and maintain the network topology. The resulting structure is a loop-free directed graph known as a Destination-Oriented Directed Acyclic Graph (DODAG), initiated by the root node. The root begins the process by broadcasting DODAG Information Object (DIO) messages to neighboring nodes to advertise the DODAG and establish upward routes. When downward routes are needed, nodes receiving a DIO join the DODAG and send a Destination Advertisement Object (DAO) either to their parent(s) or directly to the root, depending on the operating mode.

In the **storing mode**, RPL relies on hop by hop routing, where DAO messages are propagated toward parent nodes, each of which must maintain routing information for all its descendants. However, a node may be unable to accept additional children because maintaining routing table entries for its descendants exceeds its available memory resources. In such cases, the memory constrained node can reject the association, as illustrated in [15]. If the rejected node has no alternative parent, it becomes unable to join the network, a situation that is not explicitly addressed in the RPL specification. One possible solution is to modify the RPL control logic and switch to the **non-storing mode**. In this mode, only the root maintains routing state, while intermediate nodes simply forward traffic toward the root. Downward traffic is then delivered using source routing from the root to the destination.

Such a transition can be achieved through a full firmware image update on all nodes. However, large binaries lead to prolonged transmission times, higher power consumption, increased collision probability, and a greater likelihood of failures and retransmissions over bandwidth-limited multi-hop links. As shown in Fig. 1, we compare the firmware size of a full network stack (CoAP/UDP/IPv6-

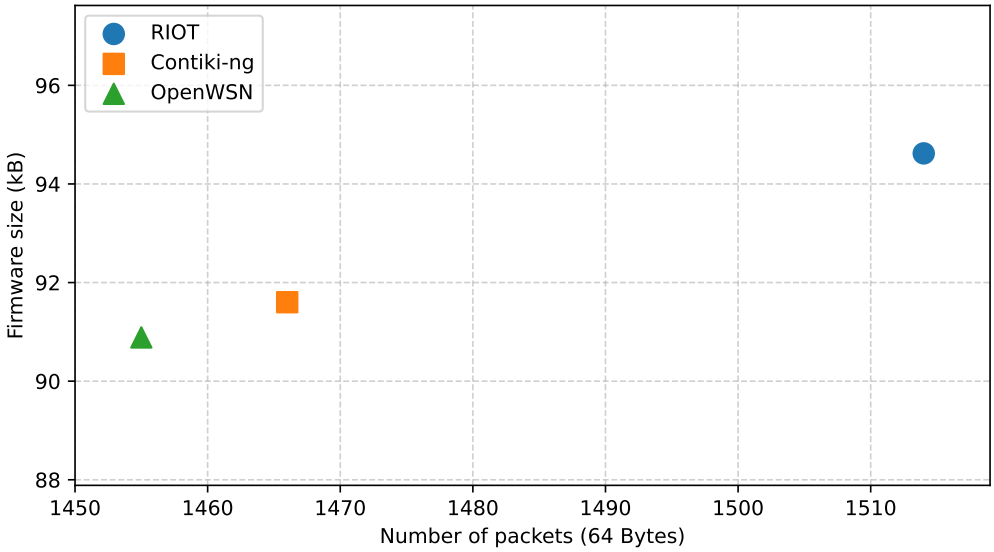

Figure 1: Firmware Size Comparison for Different Operating Systems. Firmware size of a full LLWN network stack (CoAP/UDP/IPv6-RPL/6LoWPAN/IEEE 802.15.4) in RIOT, Contiki-NG, and OpenWSN. All stacks exceed 90 kB, implying that delivering even a "minor" stack change via full-image OTA requires fragmenting the update into thousands of 64 Bytes CoAP payloads.

RPL/6LoWPAN/IEEE 802.15.4) across three major LLWN operating systems, RIOT [16], Contiki-NG [17], and OpenWSN [18]. All systems exhibit firmware sizes exceeding 90 kB, which, when segmented into 64-byte payloads (typical CoAP [19] payload size for delivering updates in LLWNs), result in thousands of packets traversing the network. This substantial traffic volume congests the already bandwidth-limited medium and significantly increases the probability of transmission failures. Consequently, minimizing update size emerges as a critical requirement for achieving efficient and reliable updates in LLWNs.

These challenges call for a modular, fine-grained network stack that enables selective, seamless updates while preserving the network state. For example, switching RPL modes should be achievable by updating only the components that exchange control messages and manage the routing state, without replacing the entire firmware or rebooting devices. Our main contribution addresses this limitation by achieving a substantial reduction in update size and time through a modular network stack that updates only the necessary components of RPL rather than replacing the full firmware image.

Beyond the large size of full-image updates, their "dumb" nature poses additional

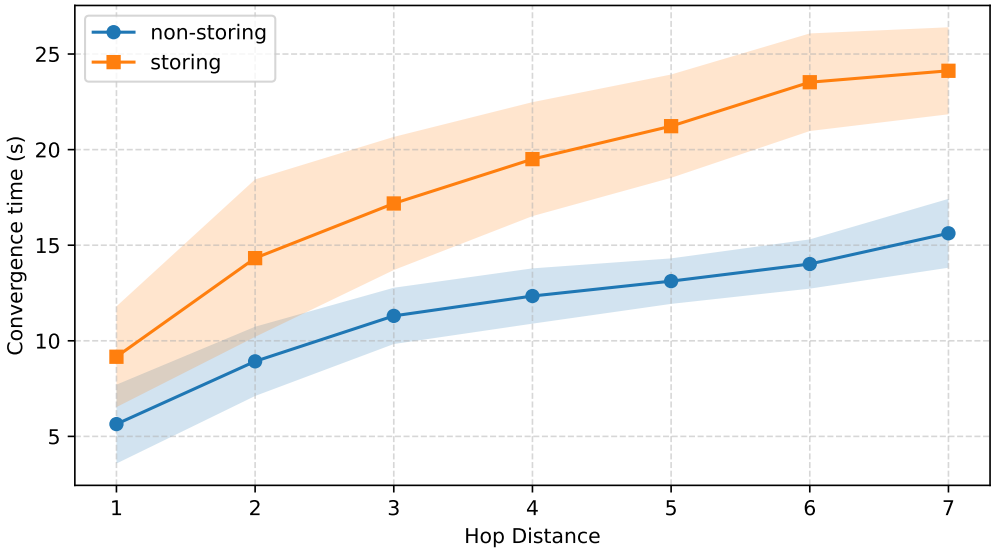

Figure 2: Convergence Time in RPL over Hop Distance in a 7-node line topology. Convergence increases with hop distance due to the sequential propagation of control messages and state installation. Non-storing mode converges slightly faster because intermediate nodes do not process/store DAO state, but both modes still exhibit multi-hop bootstrapping delays.

challenges. After a reboot, all previously built routing state is discarded, forcing RPL to initiate a new wave of control-message exchanges to reconverge. This effect is illustrated in Fig. 2, which shows the convergence time required for nodes to register at the root in a seven-node linear topology. Both storing and non-storing modes exhibit increasing convergence times as hop distance from the root grows. For instance, node 7 requires approximately 24 seconds to become reachable from the root in storing mode. Convergence is slightly faster in non-storing mode because DAOs are not processed by each intermediate node.

This network bootstrapping overhead increases with network size and hop distance from the root, leading to service interruptions, higher energy consumption, and reduced network availability. Reconvergence delays can be significantly mitigated by introducing semantics into the update process, by explicitly understanding *what* is being updated and *how* to update it safely. Centralized control-plane deployments can leverage their global network view to store state and push it back to nodes after an update, thereby avoiding reconvergence entirely.

However, fully centralized deployments, such as those in SDN, create a strong dependency on backhaul reliability. A backhaul link, which connects the nodes to

the central control plane, should reliably carry telemetry and control information across the network. Congestion or disconnections on this link can degrade telemetry collection and delay policy dissemination, leaving nodes with outdated configurations or inconsistent states. Establishing a reliable backhaul also requires dedicating resources, which reduces overall network capacity [20]. These limitations motivate the exploration of hybrid deployments, which combine the global optimization capabilities of a centralized controller, such as pushing state after updates to avoid reconvergence, with the autonomy and resilience of distributed control. This approach provides a more balanced and robust control architecture. We examine these benefits in detail in Section 4.

In summary, LLWNs face inherent challenges due to constrained resources, lossy links, and evolving application requirements. Full-image updates are inefficient and disruptive, motivating the need for programmable network protocols that allow fine-grained, semantic updates without rebooting the entire network. At the same time, different control-plane deployment models exhibit complementary strengths and weaknesses: distributed control offers resilience and autonomy but suffers from slow convergence and sub-optimal decisions, while centralized control enables global optimization but depends on reliable backhaul links. These considerations highlight the importance of a hybrid and programmable approach that combines flexible control-plane management with network-wide programmability. In the next section, we present FINE-STACK, our unified architecture that addresses these challenges.

3. The FINE-STACK Architecture

FINE-STACK introduces a modular network stack coupled with centralized orchestration. The network stack decomposes core networking functions into Virtual Functions (VFs), which can be composed to form complete network protocols. This modular design makes the stack highly fine-grained, enabling precise and efficient management of protocol components. Fine granularity is supported by remote OTA updates, allowing fast, selective, and lightweight modifications to the stack without requiring full firmware replacement. The central orchestrator, in turn, manages the network stack across nodes by updating or replacing active VFs, ensuring semantic consistency and safe updates throughout the network.

By combining modular granularity with semantically aware update mechanisms, FINE-STACK enables flexible configuration of the control and data planes, supporting hybrid and adaptive control-plane deployments. This design allows the system to dynamically switch between deployments at runtime, continuously optimizing network performance in response to evolving conditions.

3.1. Design Concepts

Our architecture is founded on four key design principles:

1. **Fine-Grained Modularity:** Instead of a monolithic stack, protocol functionality is decomposed into independently updatable services (Virtual Functions). This enables selective changes with much smaller update payloads and shorter update times, thereby reducing bandwidth usage, energy consumption, and update failure probability.
2. **Logical and Execution Isolation:** Network functions are isolated from each other and from the operating system, preventing fault propagation and simplifying lifecycle management. Isolation improves reliability, supports safe rollouts/rollbacks, and eliminates interdependencies that hinder evolution.
3. **Over-the-Air Update Capability:** The advantages of modularity are fully realized when combined with a secure and reliable remote update mechanism. Integrating OTA capabilities with a modular architecture enables semantic, efficient, and frequent updates, ensuring continuous optimization of network performance over time.
4. **Central Network Orchestrator:** A logically central orchestrator supervises the network stack across devices—spanning the data and control planes—to coordinate configuration, scheduling, and adaptation. In addition to orchestration, it can host the control plane in some deployments when global optimization is required, improving performance and policy consistency.

3.2. Network Stack

The network stack in FINE-STACK is composed of a set of lightweight Virtual Functions (VFs), which represent the minimal executable elements of the stack. Lightweight virtualization is an emerging paradigm in LLWNs that provides isolated execution environments with minimal overhead. Virtual functions in this context can be leveraged to construct modular designs, where each function encapsulates a specific functionality that can be integrated with others to perform complete tasks and updated independently. In FINE-STACK, a given networking task can be realized using a single VF or a combination of multiple VFs grouped as a submodule. These VFs can be used either to implement control-plane logic/code locally on devices in distributed control-plane deployments or to carry rules pushed remotely in centralized control-plane deployments, as illustrated in Fig. 3.

In distributed control-plane deployments, individual VFs and submodules are composed to implement complete network protocols. For example, the RPL routing protocol can be realized using VFs at the network layer as a distributed routing

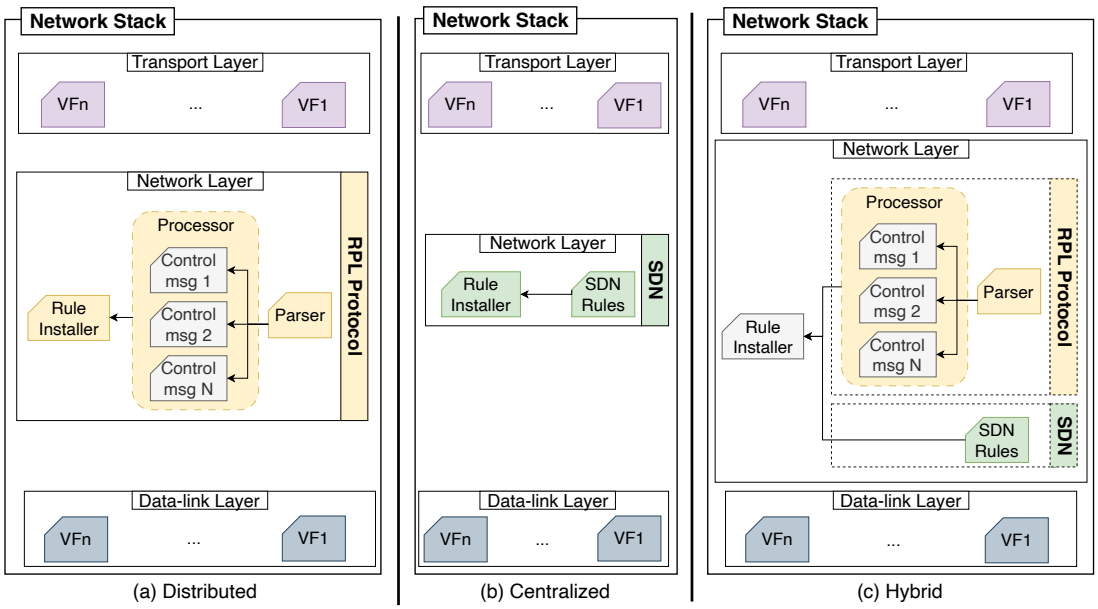

Figure 3: Network Stack in FINE-STACK is decomposed into lightweight VFs that can be composed differently depending on the control-plane deployment: (a) distributed operation where VFs implement protocol logic, (b) centralized operation where VFs primarily carry controller-generated rules to be installed locally, and (c) hybrid operation combining both.

protocol (Fig. 3-(a)). A VF may act as a parser that processes incoming packets and forwards them to a processing submodule. This submodule comprises multiple VFs, each responsible for handling a specific type of routing control message to construct the routing state, after which the output is passed to a VF acting as a rule installer to interact with the forwarding table and install the corresponding rules. The same design principle applies across other layers of the network stack.

In contrast, centralized control-plane deployments leverage VFs to transport and install rules pushed by the central control plane into the data planes of devices. For instance, an SDN-like deployment can be realized by executing VFs received from the central controller that carry new rules, which are then installed by invoking a rule installer to interact with the forwarding table (Fig. 3-(b)).

These two approaches can be combined to form a hybrid deployment (Fig. 3-(c)), in which a local control plane is implemented using VFs (e.g., the RPL protocol), while additional VFs carry rules pushed by a global centralized control plane (e.g., an SDN controller). A rule installer is then used to install rules originating from both the local and global control planes, ensuring coherent operation.

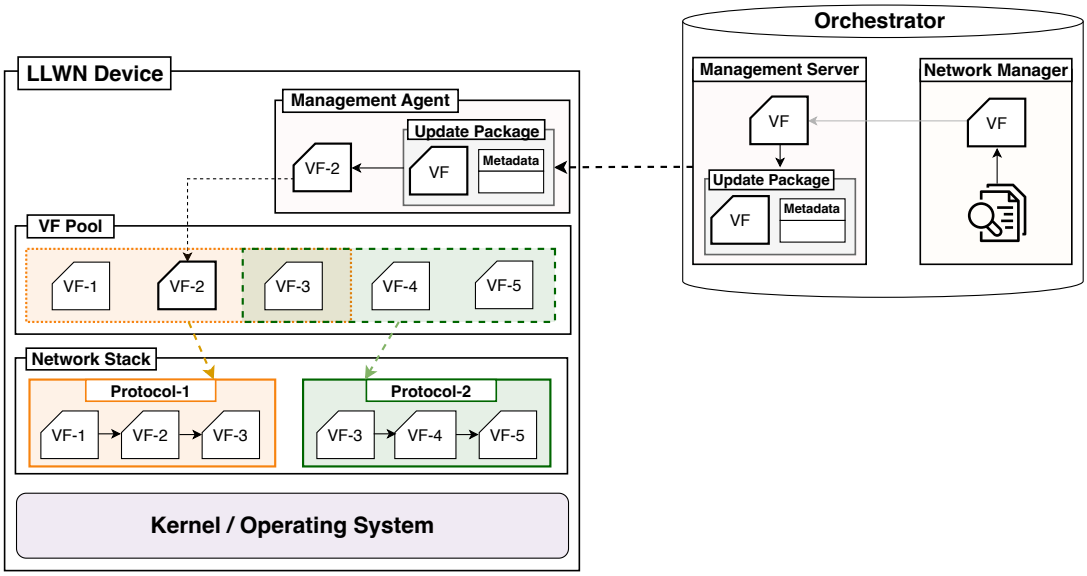

Figure 4: FINE-STACK Architecture. The orchestrator manages the VF-based network stack. Devices maintain a VF Pool and a Management Agent that validates and installs update packages, while the orchestrator hosts the Network Manager (decision/compilation) and Management Server (secure distribution) enabling semantic and coordinated updates across many constrained nodes.

Overall, the use of VFs provides substantial flexibility by allowing them to encapsulate executable logic or simply convey state rules. This versatility supports distributed, centralized, and hybrid control-plane deployments and forms the foundation of a highly programmable network stack.

The adoption of lightweight VFs is driven by the need to minimize overhead in resource-constrained LLWNs, ensuring strong isolation and independence both among VFs themselves and between VFs and the underlying operating system. This isolation guaranties **safe** and **secure** updates of network functions without compromising system stability. Constructing the network stack at the VF level introduces a high degree of modularity, enabling reuse of the same VF across different layers and protocols, thus improving **reusability** and reducing redundancy. This fine-grained modular design enables **fast**, **selective**, and **semantically** consistent runtime updates of the stack, offering an efficient and low-overhead update mechanism, especially when integrated with remote OTA management described in the following subsection.

3.3. Network Architecture

To implement the modular network stack, each LLWN device maintains a **VF Pool** that stores all locally available functions, which can be dynamically triggered by the network stack to execute specific protocol- or layer-level operations. This architecture promotes strong modularity and reusability, allowing the same VF to be invoked by multiple layers or protocols, thereby improving memory efficiency and reducing redundancy.

To manage network stacks across LLWN devices, FINE-STACK integrates a central orchestrator responsible for monitoring and configuring devices over the air, as illustrated in Fig. 4. The orchestrator comprises two main components: (i) the **Network Manager** and (ii) the **Management Server**.

The **Network Manager** is responsible for collecting network metrics, evaluating network conditions, and making adaptive decisions accordingly. These decisions may involve modifying an active VF or replacing it with a new one. Once an update is required, the Network Manager generates the new VF(s), compiles and validates them, and transfers the resulting binaries to the Management Server for distribution.

The **Management Server** serves as the orchestrator's interface to LLWN devices and ensures secure and reliable update delivery. To achieve this, each VF is encapsulated with metadata to form an *Update Package*. The metadata ensures secure delivery through authentication (using cryptographic keys) and integrity verification (using hash functions), as well as version control. It also contains installation-related information such as the target device type, update size, and the VF's designated position within the stack.

On the device side, each node hosts a **Management Agent**, which forms the endpoint of the management loop with the Management Server. Upon receiving a Update Package, the Management Agent authenticates it, verifies its integrity, and decapsulates the VF and metadata. The validated VF is then installed into the VF Pool, guided by the provided installation parameters.

The use of metadata guarantees secure and safe installation of updates, thereby mitigating risks of system failures, security breaches, and misconfigurations. Future work will focus on enhancing the intelligence of the Network Manager, particularly in its ability to collect, interpret, and leverage network metrics to determine optimal conditions and strategies for autonomous update triggering.

3.4. Update Process

The update process, illustrated in Fig. 5, follows a coordinated two-phase procedure—install then activate—to preserve network-wide consistency. It begins when the orchestrator decides to initiate an update and generates the corresponding VF

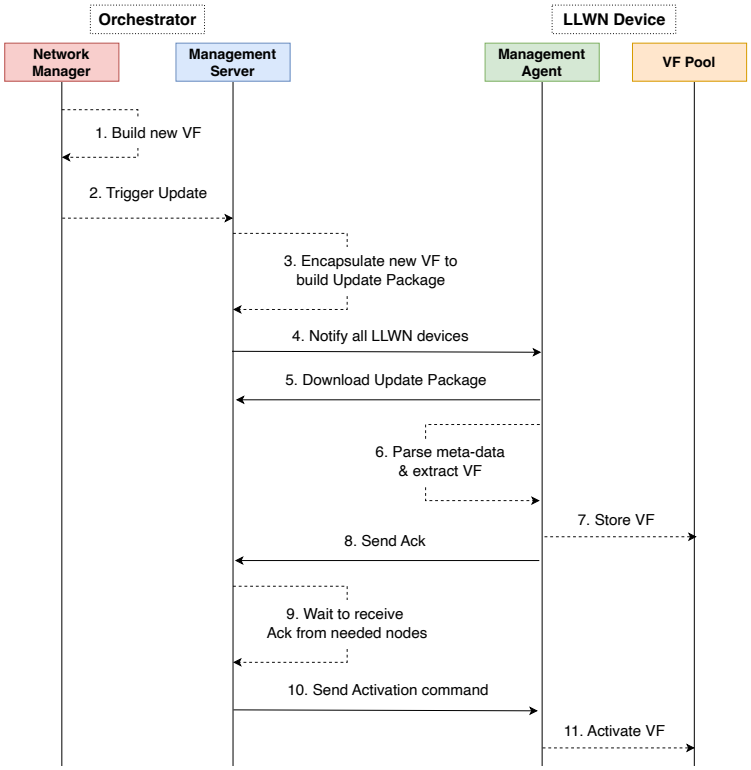

Figure 5: Update Process. Two-phase update procedure (install then activate) ensuring network-wide consistency: nodes first authenticate and store the VF in the pool, then activate it synchronously after enough nodes acknowledge installation.

binaries (1). The new VF is then transferred to the Management Server (2), which is responsible for delivering the update to the LLWN devices.

The Management Server encapsulates the VF with the required security and installation metadata to create an Update Package (3) and subsequently notifies all Management Agents in the LLWN devices about the available update (4). Each Management Agent then authenticates the orchestrator (signature verification) before initiating the download of the new VF (5). Upon successful reception, the Management Agent parses the Update Package, extracts the hash to verify integrity, retrieves the installation metadata (6), and stores the new VF in the VF Pool, awaiting an activation command (7).

Once installation is complete, each Management Agent sends an acknowledgment (Ack) message to the Management Server (8). The Management Server waits until

a sufficient number of LLWN devices confirms successful installation (9). After that, the Management Server issues an activation command to all Management Agents (10), enabling the new VF simultaneously, prompting them to activate the new VF in the pool (11). This two-phase scheme ensures that all devices run the same VF version, maintaining semantic consistency across the network.

For reliability, if an acknowledgment is not received within a specified *timeout*, the Management Server retransmits the Update Package up to a predefined *Maximum Number of Retries*. Nodes that remain unresponsive are marked as inaccessible. The activation process proceeds once the *Minimum Number of Updated Devices* threshold is reached.

Future work includes tuning retransmission parameters for update reliability under LLWN dynamics, and designing robust strategies for intermittent connectivity (e.g., staged rollouts, opportunistic retries, and delayed activation windows).

3.5. Control-plane Deployments

The modular, fine-grained design and efficient update mechanism of FINE-STACK enable flexible configuration of the network planes and flexible placement of control plane, thus supporting multiple deployment models. These include traditional distributed and centralized control-plane deployments, as well as hybrid deployments that address limitations of existing approaches (Fig. 6).

In the **distributed control-plane deployment**, the control and data planes are co-hosted on each LLWN device, enabling decentralized decision-making and autonomous operation. Typical instances include routing and distributed MAC protocols. However, this model often exhibits slow and non-optimal convergence due to localized decisions and the high volume of control traffic.

FINE-STACK realizes this deployment, as shown in Fig. 6-(a), by encapsulating both control and data planes into lightweight VFs, which can be updated over-the-air and at runtime. Unlike traditional solutions that require full-image firmware replacement for reconfiguration, FINE-STACK provides semantic and targeted updates that modify only the relevant functions. This approach results in significantly faster, more efficient, and more reliable updates.

For example, switching the RPL protocol from storing to non-storing mode (or vice versa) typically requires a complete firmware update in conventional systems—an operation that can congest the bandwidth-limited LLWN and has a high risk of failure. In contrast, FINE-STACK performs this transition seamlessly by updating only the specific VFs responsible for handling the corresponding control messages at runtime, thereby minimizing significantly update time and improving reliability.

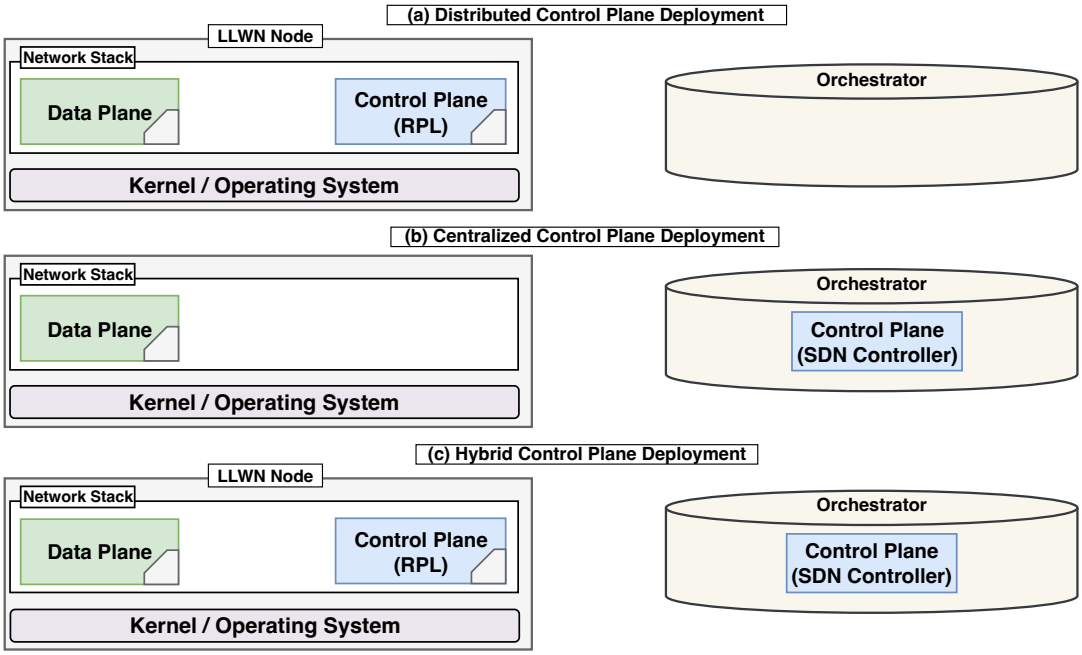

Figure 6: Supported Modes of Operation. FINE-STACK supports three control-plane placements using VF composition: (a) distributed (logic executed locally in VFs on each node), (b) centralized (controller pushes rules/state via VFs to local data planes), and (c) hybrid (local protocol autonomy plus optional global optimization/state injection).

In the **centralized control-plane deployment**, the control plane is consolidated within a central entity that globally manages the data planes of all LLWN devices. This approach increases the programmability of both control and data planes while enhancing overall network performance. Software-Defined Networking (SDN) exemplifies this deployment model, where the controller dynamically pushes rules to the data plane in devices to reconfigure the network in response to changing conditions. However, its main limitation lies in the requirement for a reliable backhaul link to maintain continuous connectivity between the distributed data planes and the central controller—an especially challenging constraint in lossy LLWN environments where latency, jitter, and link intermittency can disrupt the control loop.

FINE-STACK leverages its central orchestrator to host the control plane, which pushes rules to the data planes of LLWN devices through VFs, as shown in Fig. 6-(b). FINE-STACK enables the dissemination of rules spanning multiple layers or functional domains, thus providing a unified, multi-purpose centralized control-plane solution. Moreover, new rules can be defined and deployed through VFs without

requiring a full software update on the devices, significantly improving flexibility and reducing downtime.

For instance, like existing SDN frameworks focus on optimizing routing performance in the network layer and/or scheduling in the MAC layer, FINE-STACK can simultaneously push rules related to both. A single VF can include forwarding table updates for the network layer alongside scheduling rules for medium access in the MAC layer—allowing cross-layer optimization and finer-grained network control within a unified architecture.

A **hybrid control-plane deployment**, which combines a distributed control plane within LLWN devices and a global centralized control plane, effectively addresses the limitations of both fully distributed and centralized approaches. In this model, the global control plane is responsible for making network-wide optimization decisions, while the local control planes ensure the continuous operation of the network even when the backhaul link to the orchestrator is disrupted. Proper coordination between the global and local control planes is essential to prevent decision conflicts and maintain system coherence.

FINE-STACK enables this hybrid deployment by hosting the global control plane on the orchestrator and implementing local control planes as VFs on LLWN devices (Fig. 6-(c)). The orchestrator continues to manage and update the local control planes, as in the distributed model, while also pushing global decisions, as in the centralized model.

For instance, LLWN devices can execute RPL as a local distributed control plane, exchanging control messages to build the topology. Meanwhile, the orchestrator maintains a global view of the network, storing the state of each node (e.g., neighbor relationships). During updates, such as switching RPL modes, the orchestrator can push the stored state back to the nodes to avoid the lengthy reconvergence process that normally follows an update. This approach minimizes overhead, reduces congestion, and prevents service interruptions during topology reformation. To our knowledge, this realization of the hybrid control plane has not been demonstrated in LLWN environments, and FINE-STACK's VF-based design provides the modularity necessary to make it practical.

In conclusion, FINE-STACK unifies diverse control-plane deployments within a single coherent architecture. The usage of VFs enables a wide range of reconfiguration forms ranging from rule installation in centralized deployments to updating code and logic of running protocols in distributed deployments. Beyond improving traditional distributed and centralized approaches, it introduces a hybrid mode that combines their strengths while mitigating their respective limitations. Alongside enhanced programmability within each deployment, this unified design enables flexible and

seamless switching between deployments, thereby improving adaptability at runtime.

4. Evaluation

In this section, we implement FINE-STACK on the RIOT operating system [16] and validate it on a real-world testbed. The validation covers the three control-plane deployments introduced earlier.

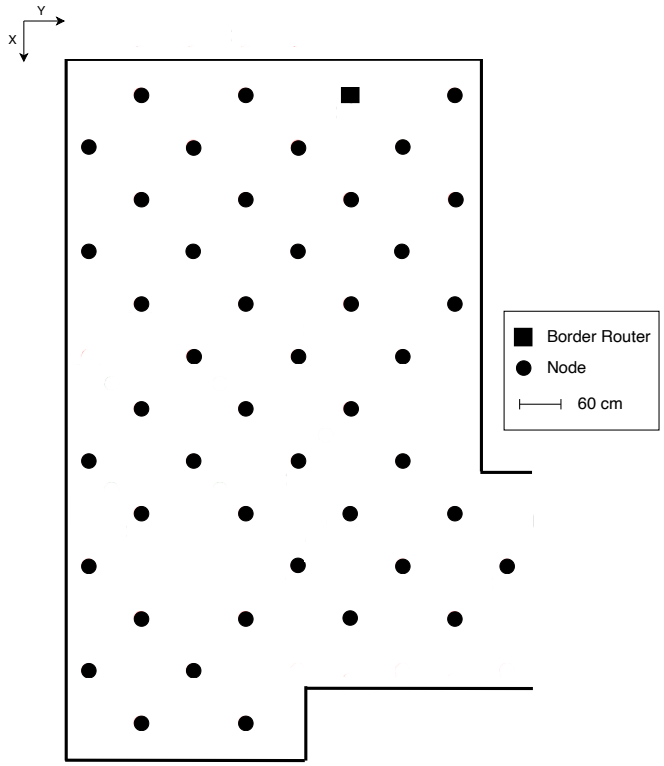

Figure 7: Topology on FIT IoT-LAB. The network comprises 46 M3 nodes (ARM Cortex-M3 MCU, 2.4 GHz IEEE 802.15.4 radio, 256 kB Flash, 64 kB RAM), forming a multi-hop topology with heterogeneous link qualities. One node serves as a Border Router (BR), providing IPv6 connectivity to the orchestrator and acting as the RPL root in the distributed deployment.

In distributed deployment, we decompose RPL into 11 VFs, to handle distinct control messages and initiate the network. We support runtime switching between

storing and non-storing modes to adapt routing behavior. Because non-storing mode is not natively available in RIOT, we implemented and contributed this feature ourselves[1]. In centralized deployment, the orchestrator acts as a central controller, installing topology rules on nodes in an SDN-like manner via VFs. The hybrid deployment represents a specific realization in which local RPL control is complemented by centrally injected state after updates. This hybrid case demonstrates how FINE-STACK can combine local protocol autonomy with global state restoration. However, general coordination between local and global control decisions remains outside the scope of this work and will be addressed in future work.

To implement our VFs, we use Femto Containers (FCs), a middleware framework designed to deploy lightweight virtual functions on constrained devices. Their lightweight nature stems from their minimal resource requirements—each FC requires only 512 bytes of memory and utilizes 11 registers, with support for memory sharing among multiple FCs. Moreover, FCs can be updated independently over-the-air, providing flexibility and modularity [21]. While the original design of FCs targets the application layer, we extend their scope to the other layers of the stack. It is important to note that our architecture is not tied to a specific virtualization technology; while we adopt Femto Containers in our implementation, other lightweight virtualization techniques can also be employed to realize the proposed VFs.

For this implementation, the Constrained Application Protocol (CoAP) [19] is employed to establish the management plane, where the Management Server and Management Agent operate as a CoAP server and client, respectively. The Software Updates for the Internet of Things (SUIT) standard [22] is used to encapsulate VF update files and metadata. The resulting management plane follows a CoAP-SUIT/UDP/IPv6 stack, ensuring compatibility with existing IoT standards and efficient operation in constrained environments.

All three deployments are evaluated using the topology shown in Fig. 7, comprising 46 nodes, including a Border Router (BR) that provides connectivity to the orchestrator and also serves as the RPL root in the distributed deployment. Experiments run on the FIT IoT-LAB real testbed [23], using M3 nodes equipped with an ARM Cortex-M3 CPU, a 2.4 GHz radio transceiver, 256 kB of Flash memory, and 64 kB of RAM.

The comparison baseline is the native RIOT stack, which mandates a full firmware reinstallation for any modification in the stack. To ensure realistic conditions, all experiments use the GoMacH duty-cycling protocol [24], employing a vTDMA-based superframe of 10 slots, each 1 ms long and capable of transmitting one packet. The

[1]https://github.com/ahmahmod/RIOT/tree/gnrc_rpl_non_storing

experiments adopt the IETF-recommended stack (IPv6/6LoWPAN/IEEE 802.15.4). However, FINE-STACK remains a general design and can support any other protocol at both the data and control planes after providing the proper implementation and validation.

A direct quantitative comparison with prior work is not feasible due to differences in supported update granularity and underlying operating systems, which strongly influence timing and memory behavior; we therefore use RIOT native updates as a baseline and a complement positioning with prior update and programmability frameworks is provided qualitatively in Section 5.1.

The evaluation focuses on three key aspects: (i) update size and time compared with the native implementation, (ii) network performance across deployments, and (iii) system-level resource utilization. To promote reproducibility, the implementation, scripts, and configurations will be released as open source[2].

4.1. Update Size and Time

This section compares how VFs are used across the different deployments, each of which features a distinct form of VF update, resulting in different update sizes and corresponding delivery times. In the centralized deployment, updates consist of rules encapsulated in VFs and installed in the data planes of LLWN devices, pushed by the central control plane running on the orchestrator. In the distributed deployment, updates modify the control-plane logic executed locally on each device through VFs to reconfigure its operation. The hybrid deployment combines both approaches, where VF updates take two distinct forms: some VFs deliver new control-plane logic to update the local control plane as in distributed deployments, while others disseminate state rules as in centralized deployments.

To measure the update time and size for each control-plane deployment, we first switch the RPL operating mode between storing and non-storing for the distributed deployment. Since this is a distributed protocol deployed on a real-life testbed, the resulting topology is unpredictable. For the centralized scenario, we instead push new forwarding rules to construct a topology that optimizes link quality over hop distance. The resulting topology consists of 22 nodes at one hop from the BR, 15 nodes at two hops, and 8 nodes at three hops, as determined by the pushed rules. Finally, in the hybrid deployment, we switch RPL between storing and non-storing modes while preserving state information to avoid costly reconvergence. Table 1 compares the update sizes across these deployments.

[2]https://github.com/ahmahmod/fine_stack

			Size (Byte)
FINE-STACK	***Centralized***		$N \times i; i \in [104, 208]$
	Distributed	*storing*	3,758
		non-storing	2,966
	Hybrid	*storing*	$3{,}758 + N \times i; i \in [104, 208]$
		non-storing	$2{,}966 + N \times i; i \in [104, 208]$
Native	***Distributed***	*storing*	123,000
		non-storing	125,728

Table 1: Update Size among Control-Plane Deployments

In the centralized control-plane deployment, the orchestrator distributes rules to network nodes to construct the topology. Each node receives a number of rules proportional to its position in the topology and the number of descendants it serves. In FINE-STACK, each rule occupies between 104 and 208 bytes, depending on whether optimizations are applied when grouping multiple rules into a single VF. The total update size for a node is therefore $N \times [104, 208]$ bytes, where N denotes the number of rules assigned to that node. In our evaluation, nodes were organized into three levels, resulting in different rule counts—and thus different update sizes—per node.

In the distributed control-plane deployment, updates correspond to switching between the two operational modes of RPL. With FINE-STACK, only 3 of the 11 VFs implementing RPL require updating, whereas the native RIOT stack mandates a full firmware image replacement. FINE-STACK achieves a dramatic reduction in update size, 96.94% when switching from non-storing to storing mode, and 97.64% in the reverse direction. These minor differences arise from structural variations in the mode-specific VF implementations. The update size is identical for all nodes, as each must update the protocol logic.

The hybrid deployment extends both approaches by retaining RPL as the local control-plane while leveraging the orchestrator as a global control plane that pushes state information to nodes. This mechanism eliminates the long reconvergence delays typically observed after RPL updates. Consequently, in addition to the 3 VFs required in the distributed deployment, the hybrid mode includes a fourth VF (the State VF), which encodes per-node state (rules). The resulting update size therefore comprises a fixed component for the distributed protocol logic and a variable component determined by the number of rules in the State VF.

The native RIOT stack, by comparison, requires a full-image update even for minor modifications. This results in very large update sizes that are impractical for bandwidth-limited networks. Additionally, device rebooting is necessary after each update, causing loss of network state and temporary interruptions in connectivity during reconvergence.

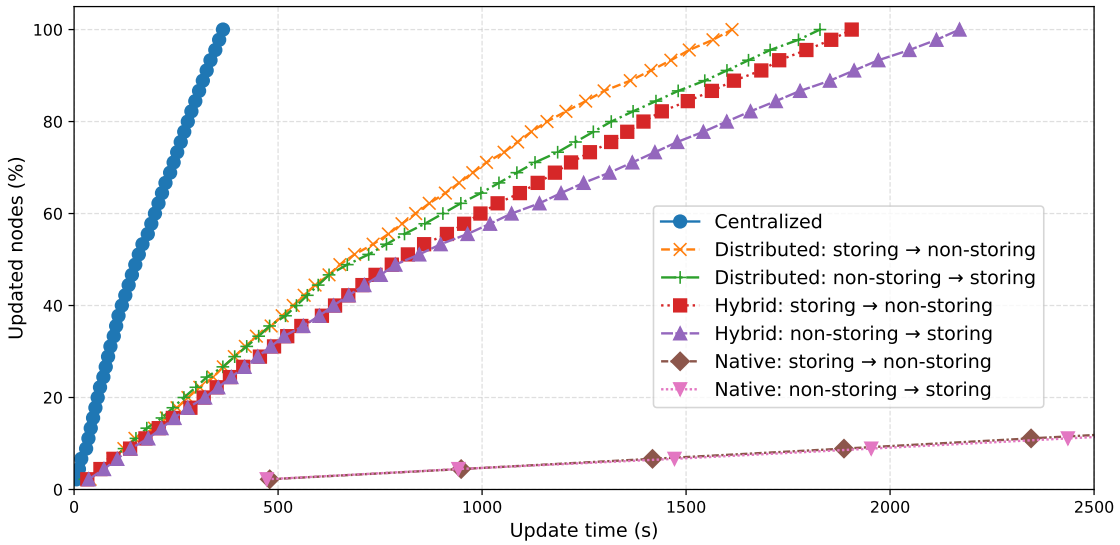

Figure 8: CDF of Node Update Times for Different Control-Plane Deployments. Centralized updates are fastest because they deliver compact rule-only VFs, while distributed updates are slower due to larger logic-bearing VFs. Hybrid updates are slowest because they combine logic updates with an additional State VF, but this overhead is still far below native full-image replacement.

The differing nature of updates across deployments leads to varying update sizes and consequently different update times. Fig. 8 presents the CDF of node update times, showing the fraction of nodes that have completed the update as a function of time. The centralized deployment achieves the shortest update time, since its VFs contain only the rules pushed from the orchestrator to the nodes. However, VFs carrying rules must be transmitted frequently to reflect changes in network conditions. In contrast, the distributed deployment requires larger updates when switching between storing and non-storing modes, as the VFs contain updated control-plane logic/code. This results in longer update times, although such updates occur far less frequently than in the centralized case. The hybrid deployment produces the longest update time because it includes the three VFs from the distributed deployment plus an additional State VF, increasing the overall update size. This overhead, however, is compensated by the elimination of reconvergence delays after updates, as discussed in the next section.

Across all deployments, FINE-STACK significantly reduces the total update time—never exceeding 3,000 seconds—compared to the native implementation, which requires full-image replacement and exceeds 27,000 seconds. This substantial improvement is achieved through FINE-STACK's fine-grained, selective, and semantic

update mechanism.

Overall, FINE-STACK drastically reduces update size and time across all deployments compared to the full-image updates of the native implementation. Minimizing update size is particularly critical in LLWNs, where bandwidth is limited and communication often relies on lossy multi-hop links. Large updates not only increase transmission time but also exacerbate congestion, packet loss, and retransmissions. While FINE-STACK significantly reduces update time, it currently relies on unicast distribution, leading to redundant transmissions in scenarios such as the distributed mode where multiple nodes receive identical updates. A promising avenue for future improvement is the integration of multicast transmission to further accelerate and optimize update dissemination.

4.2. Control-plane Deployments Performance Comparison

This section compares the advantages and limitations of each control-plane deployment and illustrates how hybrid deployments can overcome the shortcomings of both fully centralized and distributed approaches.

To evaluate convergence behavior after an update, we measured the convergence time of each node, defined as the interval until it becomes connected and reachable from the BR. Fig. 9 shows the CDF of node convergence times after update for all three deployments, based on 20 experiments. In the distributed deployment, updates switch RPL between storing and non-storing modes by updating the required VFs. After the update, nodes operate autonomously and must re-exchange control messages to rebuild their forwarding tables according to the new mode. This process results in long convergence delays, as the extensive control-message exchange both slows convergence and burdens the bandwidth-limited medium. In the centralized deployment, the network is reconfigured to use a different parent-selection metric (link quality instead of hop count) by pushing new rules to the forwarding tables of all nodes, effectively rebuilding the topology from scratch, as occurs after RPL updates. Convergence is nearly instantaneous, since nodes simply execute the VFs containing the rules pushed by the orchestrator, without requiring any interaction with neighboring nodes. In the hybrid deployment, we extend the distributed approach by switching between storing and non-storing modes while additionally pushing state information to restore the forwarding table contents. This design brings the near-instant convergence of centralized deployments to the distributed setting, effectively eliminating the lengthy reconvergence phase that typically follows an update.

While distributed deployments perform poorly in convergence after updates, centralized deployments struggle to collect metrics to maintain network view and deliver updates reliably under lossy conditions and multi-hops network. To evaluate these

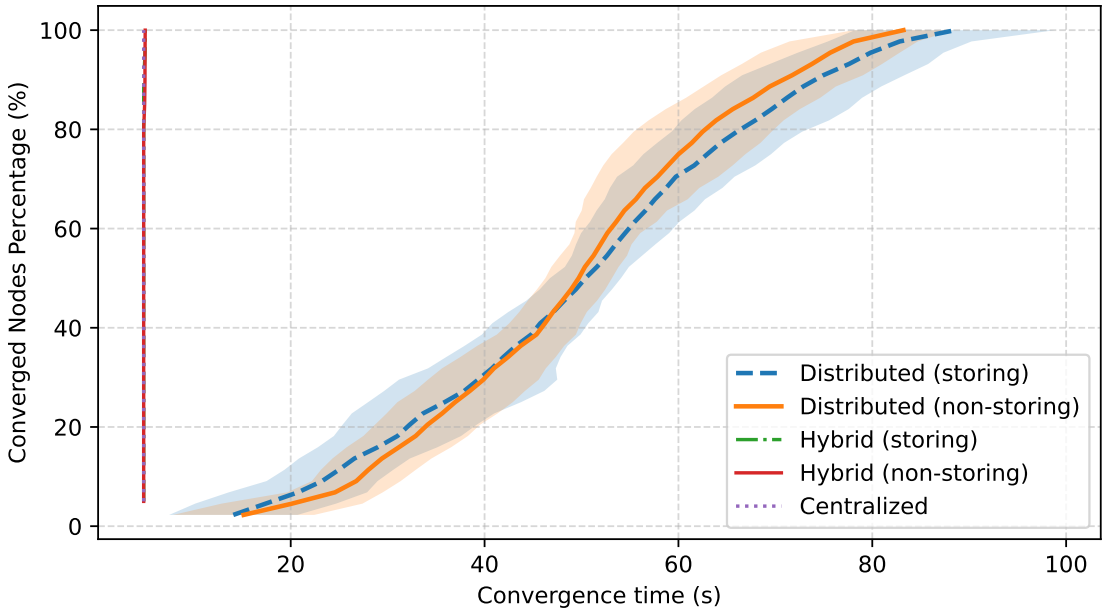

Figure 9: CDF of Node Convergence Times After Update. Distributed mode exhibits long tails because nodes must re-exchange control traffic and rebuild forwarding state after the update. Centralized mode converges almost immediately once rules are installed, since nodes do not need neighbor coordination. Hybrid mode achieves near-centralized convergence by restoring state after logic updates, illustrating that state-aware semantic updates can remove the reconvergence penalty typical of distributed protocols.

contrasting behaviors, we measured the recovery time when some nodes leave the network. In our experiments, six nodes were removed from the topology, and we measured the time required for their children to reconverge. In distributed deployment, this corresponds to the time needed for nodes to discover alternative parents; in centralized deployment, it reflects the time required for the orchestrator to recompute and disseminate new rules. The six removed nodes were selected under identical conditions in all experiments: each was a direct child of the BR, had at least two children, and each of those children had at least one descendant. To highlight the vulnerability of centralized deployments in lossy environments, we introduced network overload scenarios in which nodes periodically transmit 1-byte UDP packets (every 1 and 0.5 second) to the BR emulating common LLWN applications, such as healthcare applications [25]. Fig. 10 compares the recovery time across the three deployments under varying overload conditions, using 20 experiments per scenario.

Recovery time increases with load in all deployments. However, the centralized

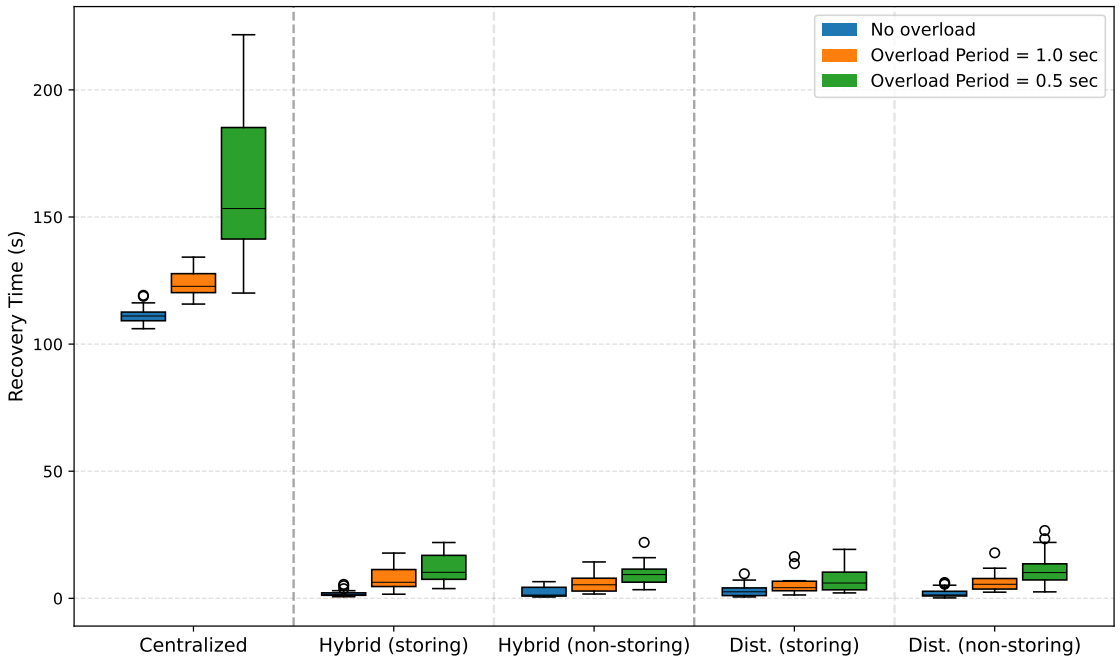

Figure 10: Maintenance Recovery Time among Control-Plane Deployments. Recovery worsens with load for all deployments, but centralized control degrades most because recomputation plus multi-hop rule dissemination is delayed by congestion and lossy links. Distributed control recovers faster via local parent reselection without relying on remote rule delivery. Hybrid control preserves this resilience by allowing local fallback while still enabling later global optimization, showing robustness under backhaul stress.

deployment experiences the longest recovery delays, as rules must be recomputed first at the orchestrator and then delivered via VFs through lossy multi-hop paths to the affected nodes. Under high load, this leads to prolonged connectivity disruption, as nodes remain disconnected while waiting for delayed reconfiguration from the central control plane. In contrast, the distributed deployment recovers more rapidly, as affected nodes can locally exchange control messages with stable neighbors and select alternative parents without relying on remote rule delivery (no VF delivery). In our experiments, nodes were configured not to cache alternative parents; enabling parent caching would allow distributed deployment to achieve near-instant recovery.

The hybrid deployment overcomes the limitations of centralized control in lossy environments by allowing nodes to fall back on their local control plane to exchange control messages and reattach to the topology, similarly to the distributed deployment. This mechanism ensures rapid reconnection and minimizes service interrup-

tion even with sub-optimal selection, while still allowing the global control plane to subsequently optimize parent selection based on its global view.

Overall, the results show that centralized and distributed deployments offer distinct advantages depending on network conditions. Centralized deployments converge immediately once updates are delivered, but they struggle with reliable update dissemination under lossy and multi-hop conditions. In contrast, distributed deployments require a reconvergence phase after updates, yet they do not depend on rule delivery from a remote controller, as illustrated in Fig. 9 and Fig. 10. The hybrid deployment consistently outperforms both approaches by combining the global optimization capabilities of centralized control with the resilience and autonomy of distributed control. A remaining challenge lies in managing potential conflicts between local and global decisions, which warrants further investigation. These findings highlight the need for flexible network architectures that support multiple deployment models and seamless runtime switching between them, while respecting the bandwidth and resource constraints of LLWNs, as discussed in the following sections.

4.3. Memory Footprint

In this section, we discuss the memory requirements across different control-plane deployments, which can be categorized into two main aspects: (i) the memory footprint introduced by the architecture and required modules, and (ii) the memory usage during the download and installation process.

4.3.1. Flash and RAM Memory

To enable FINE-STACK on the operating system, additional modules are required for the VF engine to enable VFs of the architecture and management methods needed to install new updates sent by the orchestrator. Fig. 11-(a) illustrates the flash memory footprint across the different deployments. Overall, the architecture increases the firmware size by only 7.6 kB, corresponding to 7.84% of the native RIOT firmware (96.9 kB). These modules are indispensable, as they provide the foundation for enabling the architecture in all deployments.

In the distributed and hybrid deployments, the flash memory must additionally host the RPL protocol module for the distributed control plane. The RPL implementation in FINE-STACK is approximately 1.3 times larger than RIOT's native implementation due to its decomposition into 11 VFs. This modularization introduces some unavoidable code duplication (e.g., redefining structures in each VF) and reduces compiler optimization. Nevertheless, this overhead is acceptable given the flexibility and programmability that the modular approach provides.

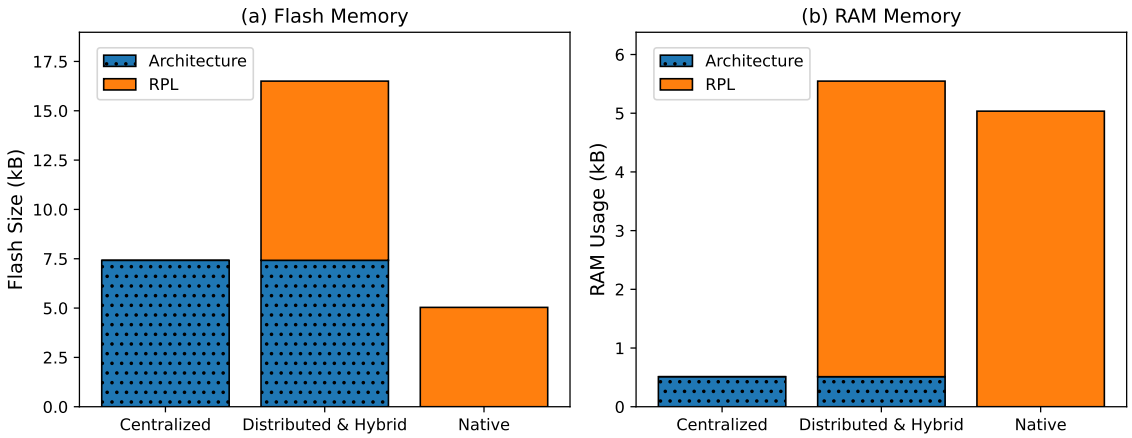

Figure 11: Memory Footprint Comparison: (a) Flash Memory, (b) RAM Memory. Flash overhead reflects the additional modules needed to enable the VF engine and management plane, while RAM overhead remains small, indicating lightweight runtime support. FINE-STACK's programmability comes with negligible RAM cost and a small flash increase relative to the baseline firmware size.

The RAM memory footprint is shown in Fig. 11-(b). The overhead is negligible, amounting to only 524 bytes required for the additional architecture modules. The RAM consumption of the RPL protocol itself remains identical between FINE-STACK and the native implementation.

In summary, the memory overhead introduced by FINE-STACK is modest and justified by the significant benefits it brings, particularly in reducing update size and time. These improvements directly enhance the programmability and adaptability of the network stack, while maintaining a resource-efficient design suitable for constrained IoT devices.

4.3.2. Download Slot

The download slot refers to the memory space required to temporarily buffer the incoming data chunks during an update. This requirement represents a critical limitation for LLWNs when using traditional full-image updates, since they demand available flash memory almost equal to the size of the running firmware. In practice, such free memory is rarely available on constrained devices, often making updates infeasible. FINE-STACK overcomes this limitation by requiring only a RAM slot equal to the size of the largest VF. For instance, in our experiments, the native implementation required a 94.4 kB or 98.1 kB slot to switch between storing and non-storing modes, respectively. By contrast, our architecture reduced this requirement to a fixed 2 kB slot. This drastic reduction not only eliminates a key barrier for

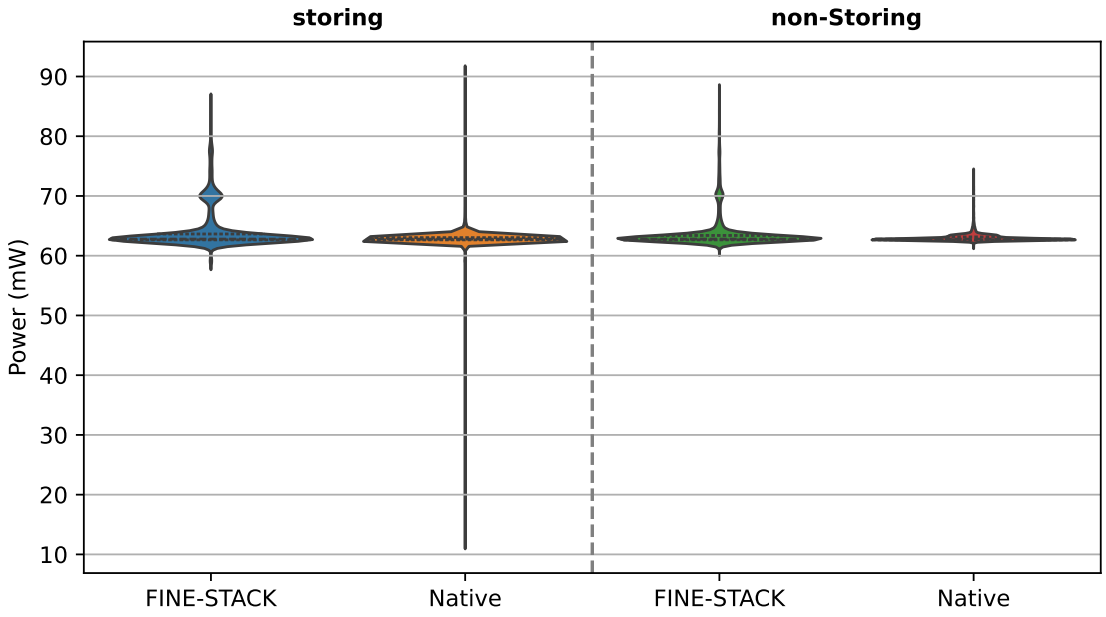

Figure 12: Power Consumption Samples Distribution. Distribution of power measurements collected in steady-state RPL operation (after 1 hour). FINE-STACK closely overlaps with the native implementation in both storing and non-storing modes, indicating that executing protocol logic through lightweight VFs does not introduce meaningful energy overhead.

updating constrained devices but also improves the practicality and reliability of in-field updates.

Overall, the memory extensions introduced by FINE-STACK remain negligible, while the elimination of excessive slot size requirements clearly demonstrates the suitability of our architecture for highly resource-constrained LLWNs.

4.4. Power Consumption

Power consumption is a primary constraint in LLWNs. To quantify the energy overhead potentially introduced by lightweight VFs, we measured the power consumption of RPL in both storing and non-storing modes under a distributed control-plane deployment, where VFs execute continuously on devices to sustain control-plane exchanges. We compare the RPL implementation of FINE-STACK with the native RIOT implementation.

For each mode and implementation, measurements were taken after one hour of continuous RPL operation, ensuring steady-state behavior with ongoing control-message exchanges. Power was monitored using the INA226 hardware component

		Update Overhead Load		Update Overhead Processing Time (msec)
		Size (bytes)	Number of Packets	
FINE-STACK	***Centralized***	$1 \times 290 = 290$	5	$1 \times 2673 = 2673$
	Distributed	$3 \times 290 = 870$	14	$3 \times 2673 = 8019$
	Hybrid	$4 \times 290 = 1160$	19	$4 \times 2673 = 10692$
Native	***Distributed***	471	8	2673

Table 2: Management overhead of updates across control-plane deployments compared to native update.

available on the FIT IoT-LAB testbed. The samples were collected periodically every 588 μs with an averaging count of 512. Fig. 12 reports the distribution of the collected power-consumption samples.

In general, FINE-STACK exhibits power consumption comparable to native implementation, reflecting the lightweight execution model of VFs. These results indicate that FINE-STACK preserves the low-energy operation required in LLWNs while enabling modularity and programmability.

4.5. Update Overhead

This section analyzes the management overhead introduced by the CoAP/SUIT-based update process in FINE-STACK and compares it with the native full-image update mechanism. In both FINE-STACK and the native implementation, the update process requires the exchange of a SUIT manifest between the orchestrator and the updated nodes. This manifest carries the metadata required for authentication, integrity verification, version control, and installation. Table 2 reports the size of the control metadata overhead, the corresponding number of transmitted packets, and the associated processing time. Note that the overhead reported in this section excludes the bytecode or binary payload corresponding to the new functionality being deployed, which is already separately reported in Table 1.

In the native full-image update, the manifest size is 471 byte, whereas in FINE-STACK each VF requires a manifest of 290 byte. This difference stems from the metadata associated with each update type: the native manifest describes a complete bootable firmware image, while the FINE-STACK manifest describes an individual VF. However, the native manifest is transmitted only once per full-image update, whereas FINE-STACK requires one manifest per updated VF. Consequently, the management overhead in FINE-STACK increases linearly with the number of updated VFs, i.e., $M \times 290$ bytes, where M denotes the number of VFs included in the update. The centralized deployment incurs the lowest management overhead, requiring only one VF to carry the rules pushed by the orchestrator, resulting in 290 bytes.

The distributed deployment requires three VFs to switch the RPL operating mode, resulting in 870 bytes. The hybrid deployment incurs the largest overhead because it updates the three RPL-related VFs and adds a fourth state VF used to restore forwarding state after the update, resulting in 1160 bytes.

Using a 64-byte CoAP payload in our evaluation, manifest transmission requires 8 control packets for the native full-image update to deliver the 471-byte manifest. In FINE-STACK, manifest transmission requires 5, 14, and 19 control packets for centralized, distributed, and hybrid deployments, respectively, corresponding to control traffic volumes of 290, 870, and 1160 bytes. This indicates that FINE-STACK does not introduce excessive signaling overhead compared to native updates. We note that no retransmissions were observed in our experimental setup; therefore, the signaling load is directly reflected by the reported control traffic.

A similar trend is observed for the processing-time overhead. Processing includes the security and installation steps required by SUIT, such as signature authentication, version verification, sequence-number validation, orchestrator-ID verification, device-class verification, and digest verification after downloading the binary file. As reported in Table 2, each update requires approximately 2673 ms of processing time, regardless of whether it corresponds to a native firmware image or a VF in FINE-STACK, excluding the reboot overhead required after native updates.

For native implementation, this processing cost is incurred only once, since the full firmware image is described by a single manifest, resulting in 2673 ms. In FINE-STACK, however, each updated VF is handled as an independent update unit and, therefore, requires its own manifest processing. Consequently, the processing overhead scales linearly with the number of updated VFs, i.e., $M \times 2673$ ms. The centralized deployment updates one VF and therefore incurs 2673 ms. The distributed deployment updates three VFs to switch the RPL mode, resulting in 8019 ms. The hybrid deployment updates the same three RPL-related VFs and an additional state VF, resulting in 10692 ms.

Overall, the increased management overhead observed in FINE-STACK is the direct cost of fine-grained modularity, where each VF is handled independently, with its own metadata, security checks, and installation process. Nevertheless, this overhead remains modest compared to the substantial reduction in update payload size and update time achieved by avoiding full-image firmware replacement.

5. Related Work

Many frameworks have explored runtime software updates for LLWN but most focus on the application layer and provide little support for updating lower layers of

the network stack, resulting in limited programmability, particularly for distributed control-plane deployments in LLWN. Early modular systems sought to decouple applications from the rest of the operating system to enable in-field updates of running applications, as in Contiki [26] and TinyOS [27]. Subsequent efforts broadened this scope by allowing applications updates with selected kernel components, improving flexibility as in SOS [28], Enix [29], RemoWare [30], and Lorien [31]. Among these, GITAR [32] is the only work that explicitly extends runtime updates to the network stack. However, its programmability remains limited in practice as it treats network protocols as kernel modules and packages each protocol as a single monolithic component rather than decomposing it into fine-grained building blocks. Consequently, GITAR offers limited granularity and semantic visibility into protocol internals (e.g., neighbor information, timers, and state machines), which hinders precise, state-aware updates and leads to inefficient network reconfiguration. In addition, some virtualization approaches have been proposed to provide stronger isolation between the application layer and the operating system, thereby enabling reliable and transparent runtime updates. Examples include WebAssembly [33] and adaptations of Java Virtual Machine for IoT environments [34]. However, despite their modularity benefits, these techniques introduce considerable runtime overhead on constrained devices, limiting both performance and the number of virtual machines that can be hosted. To overcome this limitation, lightweight virtualization has recently emerged, offering virtualized execution environments with minimal overhead suitable for constrained platforms. Notable examples include rBPF [35], which provides a practical foundation for fine-grained, on-demand updates in resource-constrained devices, and its further optimization in Femto Containers (FCs) [21], which reduces overhead even further.

The introduction of centralized control-plane deployments in LLWN have broadened programmability by separating control logic from distributed forwarding behavior. Software-Defined Networking (SDN) exemplifies this model, in which a logically centralized controller manages the data planes of the network nodes. The SDN principles have been adapted to LLWNs to optimize multiple layers and objectives. At the network layer, SDN-based approaches manage the forwarding state and improve routing performance [36, 37, 38]. At the MAC layer, SDN has been used to coordinate transmission schedules and resource allocation [39, 40]. SDN has also been used for security, enabling network-wide monitoring and mitigation of anomalous behavior through selective filtering and rate control [41, 42]. Despite these benefits, the reliance on reliable backhaul connectivity between nodes and the controller remains a fundamental limitation in LLWNs, where links are inherently lossy, intermittent, and often multi-hop. This constraint significantly hinders the practical deployment

of SDN-based solutions in real-world LLWN environments.

To mitigate the weaknesses of purely centralized or purely distributed designs, hybrid control-plane deployments have been extensively investigated, first in wired networks. A key theme is to localize decisions to reduce controller load and latency while maintaining global programmability. For example, HyperFlow [43] distributes the control state among controllers to reduce response time by handling many requests locally. Other works adopt hierarchical or partitioned control, where the network is segmented to reduce computational complexity and reroute latency, as in the hybrid hierarchical control plane proposed in [44]. The second theme focuses on the coexistence between SDN and legacy distributed routing, especially OSPF, to combine the flexibility of SDN traffic-engineering with the robustness and fault tolerance of distributed protocols. The authors in [45] and [46] use SDN-enabled nodes to partition OSPF domains into sub-domains, enabling controller-driven inter-domain optimization while keeping intra-domain routing stable. Related analyzes further emphasize the trade-off between central control and fault tolerance as the proportion of SDN-controlled nodes increases [47]. Beyond protocol coexistence, hybridization has also been explored through cloud-assisted delegation, arguing that some control and even data-plane tasks may be offloaded to remote platforms, at the cost of new latency, reliability, and security considerations [48]. Importantly, coexistence is not "free", Vissicchio et al. show that non-interacting control planes in hybrid deployments can create anomalies and provide sufficient conditions and practical guidelines to avoid disruptions during reconfiguration [49]. More broadly, they articulate the opportunities and open research challenges of hybrid SDN [50] and propose architectures in which central control augments distributed routing rather than replacing it [51].

Hybrid designs were later extended to wireless networks, largely driven by mobility, intermittency, and multi-hop constraints that make purely centralized control fragile. Abolhasan and Ni [52] argue that splitting the responsibilities between a controller and the forwarding nodes improves operability and scalability in large-scale (and potentially mobile) wireless deployments. In 5G transport contexts, METRO-HAUL highlights that practical deployments may require hierarchical control structures that blend centralized and distributed principles for resource and service management [53]. In ad hoc and mesh networks, multiple systems adopt the same fundamental strategy based on using SDN when connectivity allows, but falling back to distributed routing when the controller becomes unreachable. For example, wmSDN [54] uses a centralized OpenFlow controller for traffic engineering while relying on the OLSR routing protocol to route control traffic and maintain data forwarding in case of controller failure. Similar hybrid SDN nodes appear in MANET settings, including

Approach	Unit	Target	Network-stack Programmability	Semantics	Hybrid Control
Full-image OTA	Image	Firmware	✗	✗	✗
Application-level modular systems [26, 27, 28, 29, 30, 31]	App/module	Apps	✗	✗	✗
GITAR [32]	Protocol/module	Apps + protocols	Partial	✗	✗
General virtualization [33, 34]	VM component	Apps	✗	✗	✗
Lightweight virtualization [35]	Function	Executable logic	Partial	✗	✗
SDN-based LLWN [36, 37, 38, 39, 40, 41, 42]	Rule	Data plane	Partial	Partial	✗
Hybrid SDN in LLWN [57]	Rule + stack	Data plane + fallback	Partial	Partial	✓
FINE-STACK	**Function/rule**	**Network stack**	✓	✓	✓

Table 3: Qualitative positioning of FINE-STACK with respect to prior update and programmability approaches.

the automatic OpenFlow configuration for hybrid OpenFlow/traditional nodes [55] and heterogeneous hybrid SDN deployments where devices revert to legacy routing (e.g., OLSR) after controller disconnection [56].

In LLWNs specifically, only one recent work has introduced a hybrid control-plane approach [57]. The work improves SDN reliability by running an additional local distributed stack along the SDN stack, effectively "backing up" centralized operations when connectivity degrades. Our approach is orthogonal. Rather than extending SDN with distributed fallbacks, we extend distributed deployments with an optional global control plane that can inject stateful, semantics-aware updates and optimize decisions when appropriate. Moreover, by expressing the network stack as a set of reusable VFs, we realize hybrid control in a single unified stack, where centralized and distributed behaviors emerge from flexible VF composition rather than from maintaining two separate protocol stacks. This distinction matters in LLWNs because maintaining parallel stacks not only increases memory and complexity overhead, but also limits semantic update granularity: our VF-level decomposition enables targeted, state-preserving adaptations (e.g., tuning neighbor handling, timer logic, or scheduling policies) without redeploying an entire protocol module or switching between duplicated implementations.

5.1. Positioning with Prior Approaches

Table 3 highlights that FINE-STACK differs from prior update frameworks in both the target and the granularity of the programmability. Compared with application-level update systems, FINE-STACK explicitly targets the network stack and decomposes protocol behavior into smaller VFs that can be updated independently. Compared with GITAR, which supports network-stack updates at the protocol/module level, FINE-STACK exposes finer semantic units inside the protocol logic, enabling targeted updates of only the affected functions. Compared with application virtualization systems, FINE-STACK is not only an execution substrate but a complete network-stack architecture that defines how VFs are composed, managed, updated, and coordinated through an orchestrator.

FINE-STACK also differs from SDN-based LLWN approaches. SDN improves programmability by centralizing decision-making and pushing rules to constrained devices, but this model remains dependent on the reliability of the controller-to-node communication path. In contrast, FINE-STACK can operate as a centralized system when rule-based control is appropriate, as a distributed system when local autonomy is needed, or as a hybrid system that combines both. This is particularly important for LLWNs, where lossy multi-hop connectivity can delay or prevent centralized rule delivery. Finally, unlike hybrid SDN approaches that often maintain separate centralized and distributed stack, FINE-STACK realizes hybrid behavior through VF composition inside a unified stack. This enables state-aware updates, such as switching RPL modes while restoring routing state, without requiring full firmware replacement or maintaining duplicated protocol implementations.

6. Conclusion and Future Work

Low-power and Lossy Wireless Networks (LLWNs) should evolve after deployment to accommodate changing application requirements, network conditions, and protocol limitations. Updating LLWN nodes is therefore unavoidable, but existing approaches based on full firmware image replacement are costly, disruptive, and poorly suited to constrained devices. In this article, we introduced FINE-STACK, a unified and adaptive network architecture that enables fine-grained, semantic updates of the network stack while supporting distributed, centralized, and hybrid control-plane deployments. At the core of FINE-STACK lies a virtual function abstraction that decomposes network protocols into small, reusable functional blocks. Chaining these virtual functions allows the protocol behavior to be modified incrementally, enabling targeted updates without reprogramming the entire network stack.

We implemented FINE-STACK on the RIOT operating system and evaluated it on the FIT IoT-LAB testbed. We selected RPL as a representative baseline protocol

for LLWNs to demonstrate how FINE-STACK enables efficient and safe updates of control-plane logic in practice. Obviously, FINE-STACK is not limited to RPL and could be used to implement other network protocols as well. Through extensive evaluation, we showed that FINE-STACK significantly reduces update overhead, which in turn shortens update time. This efficiency allows the network to be updated more frequently without disrupting the applications. We also proposed a hybrid control-plane deployment that combines the benefits of centralized approaches, which provide globally optimized network decisions, with those of distributed approaches, which enable rapid reactions to local changes. By preserving the state of the network during updates, this hybrid mode avoids costly reconvergence and improves network availability, allowing nodes to reach a stable state even if the central controller becomes temporarily unreachable. Finally, we demonstrate that FINE-STACK has a minor or no impact on energy consumption, memory usage and control overhead compared to the native network stack implementation in RIOT.

Looking beyond RPL, FINE-STACK is designed to support multiple network protocols, demonstrating its flexibility and broad applicability. Its virtual function-based design enables targeted incremental updates and allows developers to compose and modify network protocol logic at runtime without full firmware replacements. By providing this modular and programmable approach, FINE-STACK paves the way for the design of future network protocols that are flexible, resilient, and capable of evolving dynamically after deployment.

In future work, FINE-STACK can be extended to support additional protocols and larger-scale deployments, leveraging simulation to assess scalability and performance limits. It can also be integrated with emerging IoT standards (e.g., Wi-Fi HaLow or Thread 1.4) to further enhance adaptability and overall network performance. We also plan to investigate efficient multicast-based dissemination mechanisms for distributing updates in multi-hop networks, as well as coordination strategies that harmonize global and local rules to prevent decision conflicts. Furthermore, we plan to integrate FINE-STACK with digital twins to explore intelligent decision engines that determine when and what updates to apply to the network based on current and predicted conditions. The digital twin will allow updates to be tested and validated before deployment in the real network, improving safety, reliability, and overall network performance.

References

[1] Y.-K. Chen, Challenges and opportunities of internet of things, in: 17th Asia and South Pacific Design Automation Conference, 2012, pp. 383–388.

[2] J. W. Hui, D. E. Culler, Extending IP to low-power, wireless personal area networks, IEEE Internet Computing 12 (4) (2008) 37–45.

[3] P. Ruckebusch, S. Giannoulis, I. Moerman, J. Hoebeke, E. De Poorter, Modelling the energy consumption for over-the-air software updates in LPWAN networks: SigFox, LoRa and IEEE 802.15.4g, Elsevier Internet of Things 3-4 (2018) 104–119.

[4] K. Zandberg, K. Schleiser, F. Acosta, H. Tschofenig, E. Baccelli, Secure firmware updates for constrained iot devices using open standards: A reality check, IEEE access 7 (2019) 71907–71920.

[5] H. Petersen, M. Lenders, M. Wählisch, O. Hahm, E. Baccelli, Old Wine in New Skins? Revisiting the Software Architecture for IP Network Stacks on Constrained IoT Devices, in: Proceedings of the 2015 Workshop on IoT Challenges in Mobile and Industrial Systems, Association for Computing Machinery, 2015, p. 31–35.

[6] J. Bauwens, P. Ruckebusch, S. Giannoulis, I. Moerman, E. D. Poorter, Over-the-Air Software Updates in the Internet of Things: An Overview of Key Principles, IEEE Communications Magazine 58 (2) (2020) 35–41.

[7] H. Mostafaei, M. Menth, Software-defined wireless sensor networks: A survey, Journal of Network and Computer Applications 119 (2018) 42–56.

[8] D. Kreutz, F. M. Ramos, P. E. Verissimo, C. E. Rothenberg, S. Azodolmolky, S. Uhlig, Software-defined networking: A comprehensive survey, Proceedings of the IEEE 103 (1) (2014) 14–76.

[9] O. Gaddour, A. Koubâa, RPL in a nutshell: A survey, Computer Networks 56 (14) (2012) 3163–3178.

[10] IEEE Standard for Local and metropolitan area networks–Part 15.4: Low-Rate Wireless Personal Area Networks (LR-WPANs), IEEE Std 802.15.4-2011 (Revision of IEEE Std 802.15.4-2006) (2011) 1–314.

[11] I. Rabet, S. P. Selvaraju, H. Fotouhi, M. Alves, M. Vahabi, A. Balador, M. Björkman, SDMob: SDN-based mobility management for IoT networks, Journal of Sensor and Actuator Networks 11 (1) (2022) 8.

[12] M. Baddeley, R. Nejabati, G. Oikonomou, M. Sooriyabandara, D. Simeonidou, Evolving SDN for low-power IoT networks, in: 2018 4th IEEE Conference on Network Softwarization and Workshops (NetSoft), IEEE, 2018, pp. 71–79.

[13] F. Veisi, J. Montavont, F. Theoleyre, Link Quality Estimation in Wireless Software Defined Network with a Reliable Control Plane, in: 2023 IEEE 9th International Conference on Network Softwarization (NetSoft), IEEE, 2023.

[14] A. Mahmod, J. Montavont, T. Noel, Pensil: Programmable network stack for low-power lossy iot networks using lightweight-virtualization, Internet of Things (2025) 101829.

[15] T. Istomin, O. Iova, G. P. Picco, C. Kiraly, Route or Flood? Reliable and Efficient Support for Downward Traffic in RPL, ACM Transactions on Sensor Networks 16 (1) (2020).

[16] E. Baccelli, C. Gündoğan, O. Hahm, P. Kietzmann, M. S. Lenders, H. Petersen, K. Schleiser, T. C. Schmidt, M. Wählisch, RIOT: An open source operating system for low-end embedded devices in the IoT, IEEE Internet of Things Journal 5 (6) (2018) 4428–4440.

[17] G. Oikonomou, S. Duquennoy, A. Elsts, J. Eriksson, Y. Tanaka, N. Tsiftes, The Contiki-NG open source operating system for next generation IoT devices, SoftwareX 18 (2022) 101089.

[18] T. Watteyne, X. Vilajosana, B. Kerkez, F. Chraim, K. Weekly, Q. Wang, S. Glaser, K. Pister, OpenWSN: a standards-based low-power wireless development environment, Transactions on Emerging Telecommunications Technologies 23 (5) (2012) 480–493.

[19] C. Bormann, A. P. Castellani, Z. Shelby, CoAP: An Application Protocol for Billions of Tiny Internet Nodes, IEEE Internet Computing 16 (2) (2012) 62–67.

[20] F. Veisi, J. Montavont, F. Theoleyre, Link Quality Estimation in Wireless Software Defined Network with a Reliable Control Plane, in: 2023 IEEE 9th International Conference on Network Softwarization (NetSoft), IEEE, 2023.

[21] K. Zandberg, E. Baccelli, S. Yuan, F. Besson, J.-P. Talpin, Femto-containers: lightweight virtualization and fault isolation for small software functions on low-power IoT microcontrollers, in: Proceedings of the 23rd ACM/IFIP International Middleware Conference, 2022, pp. 161–173.

[22] B. Moran, H. Tschofenig, D. Brown, M. Meriac, A Firmware Update Architecture for Internet of Things, RFC 9019 (Apr. 2021).

[23] C. Adjih, E. Baccelli, E. Fleury, G. Harter, N. Mitton, T. Noel, R. Pissard-Gibollet, F. Saint-Marcel, G. Schreiner, J. Vandaele, et al., FIT IoT-LAB: A large scale open experimental IoT testbed, in: 2015 IEEE 2nd World Forum on Internet of Things (WF-IoT), IEEE, 2015, pp. 459–464.

[24] S. Zhuo, Y.-Q. Song, GoMacH: A traffic adaptive multi-channel MAC protocol for IoT, in: 2017 IEEE 42nd Conference on Local Computer Networks (LCN), IEEE, 2017, pp. 489–497.

[25] A. Elsts, X. Fafoutis, G. Oikonomou, R. Piechocki, I. Craddock, TSCH networks for health IoT: Design, evaluation, and trials in the wild, ACM Transactions on Internet of Things 1 (2) (2020) 1–27.

[26] A. Dunkels, N. Finne, J. Eriksson, T. Voigt, Run-time dynamic linking for reprogramming wireless sensor networks, in: Proceedings of the 4th International Conference on Embedded Networked Sensor Systems, SenSys '06, Association for Computing Machinery, 2006, p. 15–28.

[27] W. Munawar, M. H. Alizai, O. Landsiedel, K. Wehrle, Dynamic TinyOS: Modular and Transparent Incremental Code-Updates for Sensor Networks, in: 2010 IEEE International Conference on Communications, 2010, pp. 1–6.

[28] C.-C. Han, R. Kumar, R. Shea, E. Kohler, M. Srivastava, A dynamic operating system for sensor nodes, in: Proceedings of the 3rd International Conference on Mobile Systems, Applications, and Services, MobiSys '05, Association for Computing Machinery, 2005, p. 163–176.

[29] Y.-T. Chen, T.-C. Chien, P. H. Chou, Enix: a lightweight dynamic operating system for tightly constrained wireless sensor platforms, in: Proceedings of the 8th ACM Conference on Embedded Networked Sensor Systems, 2010, pp. 183–196.

[30] A. Taherkordi, F. Loiret, R. Rouvoy, F. Eliassen, Optimizing sensor network reprogramming via in situ reconfigurable components, ACM Transactions on Sensor Networks (TOSN) 9 (2) (2013) 1–33.

[31] B. Porter, U. Roedig, G. Coulson, Type-safe updating for modular WSN software, in: 2011 International Conference on Distributed Computing in Sensor Systems and Workshops (DCOSS), IEEE, 2011, pp. 1–8.

[32] P. Ruckebusch, E. De Poorter, C. Fortuna, I. Moerman, GITAR: Generic extension for Internet-of-Things ARchitectures enabling dynamic updates of network and application modules Author links open overlay panel, Ad Hoc Networks 36 (2016) 127–151.

[33] A. Haas, A. Rossberg, D. L. Schuff, B. L. Titzer, M. Holman, D. Gohman, L. Wagner, A. Zakai, J. Bastien, Bringing the web up to speed with WebAssembly, in: Proceedings of the 38th ACM SIGPLAN Conference on Programming Language Design and Implementation, Association for Computing Machinery, 2017, p. 185–200.

[34] N. Brouwers, K. Langendoen, P. Corke, Darjeeling, a feature-rich VM for the resource poor, in: Proceedings of the 7th ACM Conference on Embedded Networked Sensor Systems, 2009, pp. 169–182.

[35] K. Zandberg, E. Baccelli, Minimal Virtual Machines on IoT Microcontrollers: The Case of Berkeley Packet Filters with rBPF, in: 2020 9th IFIP International Conference on Performance Evaluation and Modeling in Wireless Networks (PEMWN), 2020, pp. 1–6.

[36] L. Galluccio, S. Milardo, G. Morabito, S. Palazzo, SDN-WISE: Design, prototyping and experimentation of a stateful SDN solution for WIreless SEnsor networks, in: 2015 IEEE Conference on Computer Communications (INFOCOM), 2015, pp. 513–521.

[37] R. Samadi, A. Nazari, J. Seitz, Intelligent Energy-Aware Routing Protocol in Mobile IoT Networks Based on SDN, IEEE Transactions on Green Communications and Networking 7 (4) (2023) 2093–2103.

[38] A. Ouhab, T. Abreu, H. Slimani, A. Mellouk, Energy-efficient clustering and routing algorithm for large-scale SDN-based IoT monitoring, in: ICC 2020-2020 IEEE International Conference on Communications (ICC), IEEE, 2020, pp. 1–6.

[39] F. Veisi, J. Montavont, F. Theoleyre, Enabling Centralized Scheduling Using Software Defined Networking in Industrial Wireless Sensor Networks, IEEE Internet of Things Journal 10 (23) (2023) 20675–20685.

[40] M. Baddeley, R. Nejabati, G. Oikonomou, S. Gormus, M. Sooriyabandara, D. Simeonidou, Isolating SDN control traffic with layer-2 slicing in 6TiSCH industrial IoT networks, in: 2017 IEEE Conference on Network Function Virtualization and Software Defined Networks (NFV-SDN), IEEE, 2017, pp. 247–251.

[41] Y. Zhou, G. Cheng, S. Yu, An SDN-Enabled Proactive Defense Framework for DDoS Mitigation in IoT Networks, IEEE Transactions on Information Forensics and Security 16 (2021) 5366–5380.

[42] U. H. Garba, A. N. Toosi, M. F. Pasha, S. Khan, SDN-based detection and mitigation of DDoS attacks on smart homes, Computer Communications 221 (2024) 29–41.

[43] A. Tootoonchian, Y. Ganjali, HyperFlow: A Distributed Control Plane for OpenFlow, in: Proceedings of the 2010 internet network management conference on Research on enterprise networking, Vol. 3, 2010, p. 6.

[44] Y. Fu, J. Bi, Z. Chen, K. Gao, B. Zhang, G. Chen, J. Wu, A Hybrid Hierarchical Control Plane for Flow-Based Large-Scale Software-Defined Networks, IEEE Transactions on Network and Service Management 12 (2) (2015) 117–131.

[45] M. Caria, A. Jukan, M. Hoffmann, SDN Partitioning: A Centralized Control Plane for Distributed Routing Protocols, IEEE Transactions on Network and Service Management 13 (3) (2016) 381–393.

[46] M. Caria, T. Das, A. Jukan, M. Hoffmann, Divide and conquer: Partitioning OSPF networks with SDN, in: 2015 IFIP/IEEE International Symposium on Integrated Network Management (IM), 2015, pp. 467–474.

[47] A. A. Khan, M. Zafrullah, M. Hussain, A. Ahmad, Performance analysis of OSPF and hybrid networks, in: 2017 International Symposium on Wireless Systems and Networks (ISWSN), IEEE, 2017, pp. 1–4.

[48] P. K. Dey, M. Yuksel, Hybrid Cloud Integration of Routing Control & Data Planes, in: Proceedings of the 2016 ACM Workshop on Cloud-Assisted Networking, Association for Computing Machinery, 2016, p. 25–30.

[49] S. Vissicchio, L. Cittadini, O. Bonaventure, G. G. Xie, L. Vanbever, On the coexistence of distributed and centralized routing control-planes, in: 2015 IEEE Conference on Computer Communications (INFOCOM), IEEE, 2015, pp. 469–477.

[50] S. Vissicchio, L. Vanbever, O. Bonaventure, Opportunities and research challenges of hybrid software defined networks, ACM SIGCOMM Computer Communication Review 44 (2) (2014) 70–75.

[51] S. Vissicchio, O. Tilmans, L. Vanbever, J. Rexford, Central Control Over Distributed Routing, in: Proceedings of the 2015 ACM Conference on Special Interest Group on Data Communication, SIGCOMM '15, Association for Computing Machinery, 2015, p. 43–56.

[52] M. Abolhasan, J. Lipman, W. Ni, B. Hagelstein, Software-defined wireless networking: centralized, distributed, or hybrid?, IEEE Network 29 (4) (2015) 32–38.

[53] D. King, A. Farrel, E. N. King, R. Casellas, L. Velasco, R. Nejabati, A. Lord, The dichotomy of distributed and centralized control: METRO-HAUL, when control planes collide for 5G networks, Optical Switching and Networking 33 (2019) 49–55.

[54] A. Detti, C. Pisa, S. Salsano, N. Blefari-Melazzi, Wireless mesh software defined networks (wmsdn), in: 2013 IEEE 9th International Conference on Wireless and Mobile Computing, Networking and Communications (WiMob), 2013, pp. 89–95.

[55] S. Sharma, A. Nag, P. Stynes, M. Nekovee, Automatic configuration of openflow in wireless mobile ad hoc networks, in: 2019 International Conference on High Performance Computing & Simulation (HPCS), 2019, pp. 367–373.

[56] D. Sousa, S. Sargento, M. Luís, Hybrid wireless network with sdn and legacy devices in ad-hoc environments, in: 2022 13th International Conference on Network of the Future (NoF), 2022, pp. 1–9.

[57] M. Bonanni, F. Chiti, L. Pierucci, Hybrid Dual Stack Control Plane Design for Resilient Software-Defined WSANs, IEEE Internet of Things Journal 12 (18) (2025) 37853–37862.